The Beginner's Guide to Docker Networking

Understanding Docker networks is the secret to making your multi-container applications talk to each other securely and reliably. Without this foundation, you will constantly fight connection errors and expose your applications to security risks.

What you'll learn:

1. Creating Custom Bridge Networks
2. Troubleshooting with Network Inspect
3. Connecting Containers on the Fly
4. Cleaning Up Your Network Space

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Creating Custom Bridge Networks

By default, Docker puts all containers on a shared default network where they cannot find each other by name. Creating a custom bridge network enables automatic DNS resolution inside Docker. This means your frontend container can talk to your database container simply by using the database container's name.

```
$docker network create my-custom-net
 docker run -d --name my-db --network my-custom-net postgres
 docker run -d --name my-web --network my-custom-net -p 80:80 nginx
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Troubleshooting with Network Inspect

When containers fail to communicate, you need to see what is happening under the hood. The network inspect command shows you a detailed list of all connected containers, their gateway configurations, and their assigned IP addresses. This is your primary tool for debugging network issues.

```
$docker network inspect my-custom-net
```

3

TIP 3 OF 4

Connecting Containers on the Fly

You do not need to stop a running container to change its network. Docker allows you to dynamically attach and detach running containers to different networks. This is incredibly useful for connecting existing applications to new services without causing any downtime.

```
$docker network connect my-custom-net my-running-app
 docker network disconnect bridge my-running-app
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Cleaning Up Your Network Space

Every time you run a new docker-compose file or follow a tutorial, Docker creates new networks. Over time, these unused networks pile up and clutter your system. Running a simple prune command safely removes all networks that are not actively being used by at least one container.

```
$docker network prune -f
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always use custom bridge networks instead of the default bridge network.
+ Use container names instead of IP addresses in your application configuration files.
+ Isolate different applications by placing them on completely separate networks.
+ Keep backend databases off public-facing networks to protect your data.
+ Run network prune regularly to keep your development environment clean.

Watch Out For

! Trying to use container names to talk to each other on the default bridge network (it only works on custom networks).
! Hardcoding container IP addresses which change every time a container restarts.
! Exposing database ports to your host machine when only other containers need access to them.
! Using host networking by default, which bypasses security isolation and causes port conflicts.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh